Звіт з лабораторної роботи №4 (Варіант 5)

1. Інформація про виконавця: Виконав студент К-24 Остапенко Роман. Компілятор: GCC (g++ 16.2.0, MSYS2 UCRT64), режим збирання Release з оптимізацією -O3.

2. Умова задачі (специфічна частина Варіанту №5): Розробити потокобезпечну структуру даних із m = 3 цілих полів (Поле 0, Поле 1, Поле 2) з операціями читання і запису кожного поля окремо та перевантаженим operator string для всієї структури без використання атомарних змінних, векторів та масивів (окрім дозволеного звичайного масиву розміру m = 3 всередині структури). Структура оптимізується під такий розподіл частот операцій: Поле 0: read = 9%, write = 1%; Поле 1: read = 40%, write = 10%; Поле 2: read = 10%, write = 10%; запит рядка string: 20% (разом 100%).

3. Схема захисту даних та її обґрунтування: У класі OptimizedData використано масив int fields_[3] та 3 окремі м'ютекси типу std::shared_mutex (mtx0_ захищає fields_[0], mtx1_ захищає fields_[1], mtx2_ захищає fields_[2]), кожен із яких вирівняно за межею кеш-лінії (alignas(64)) для усунення хибного поділу кешу (false sharing) між ядрами CPU. Обґрунтування: по-перше, 80% операцій звертаються лише до одного конкретного поля, тому наявність 3 незалежних м'ютексів (замість одного загального) дозволяє потокам паралельно працювати з різними полями без взаємного блокування. По друге, оскільки операція operator string() (20%) читає всі 3 поля, фактична частка звернень на читання для кожного поля значно переважає запис: для Поля 0: 29% читань (9% read 0 + 20% string) проти 1% запису (29:1); для найбільш навантаженого Поля 1 -60% читань (40% read 1 + 20% string) проти 10% запису (співвідношення 6:1); для Поля 2: 30% читань (10% read 2 + 20% string) проти 10% запису (3:1). Завдяки застосуванню std::shared_mutex операції get(i) та operator string() захоплюють спільне блокування читання (std::shared_lock) і виконуються в різних потоках паралельно без очікування, а std::lock_guard береться лише під час запису set(i, val) (21% усіх дій - за умовою). В operator string() const 3 спільні блокування захоплюються у строго фіксованому канонічному порядку (mtx0_ --> mtx1_ --> mtx2_) лише на момент копіювання значень 3-ьох полів у локальні змінні, після чого м'ютекси одразу звільняються до виклику std::format. Оскільки це єдина операція, що бере більше одного м'ютекса, і порядок їх захоплення фіксований, взаємоблокування (deadlock) неможливе, а тривалість критичної секції мінімальна.

4. Таблиця розмірів 3×3 з усередненими результатами виконання: Для кожного з 3 сценаріїв згенеровано окремі файли для кожного потоку при фіксованій загальній кількості N_total = 600 тис операцій (1 потік: 1×600 тис.; 2 потоки: 2×300 тис.; 3 потоки: 3×200 тис.). Файли зчитувалися у пам'ять (std::string) до старту таймера std::chrono::steady_clock. Наведено середній час (мс) за 5 прогонами у режимі Release (-O3):

Послідовність дій (розподіл частот операцій)	1 потік (1 × 600 000 дій)	2 потоки (2 × 300 000 дій)	3 потоки (3 × 200 000 дій)
а) Відповідає умові Варіанту 5 (r0=9%, w0=1%, r1=40%, w1=10%, r2=10%, w2=10%, string=20%)	**116.43 мс**	**304.33 мс**	**323.24 мс**
б) Усі частоти рівні (усі 7 операцій рівноймовірні: по 1/7 ≈ 14.29% кожна)	**105.56 мс**	**286.64 мс**	**290.84 мс**
в) Не відповідає умові і не є рівними (string=50%, w0=30%, w2=9%, w1=5%, r2=3%, r0=2%, r1=1%)	**187.38 мс**	**514.73 мс**	**496.54 мс**

5. Висновки (відповідність результатів очікуванням): Отримані результати повністю відповідають теоретичним очікуванням. По перше, сценарій в (50% операцій string, що блокують усі 3 м'ютекси одночасно, та 44% ексклюзивних записів write, зокрема 30% записів у Поле 0 замість 1% за умовою) є найповільнішим у всіх режимах (187.38 / 514.73 / 496.54 мс - у 1.6 раза довше за сценарії а та б), бо часті ексклюзивні блокування постійно переривають спільний доступ string. По -друге, сценарій б) виконується незначно (+- 9%) швидше за сценарій а через два об'єктивні чинники: у сценарії б менша частка найдорожчої трим'ютексної операції string (14.29% проти 20% в а), тобто на 34 286 менше форматувань рядків і в середньому 1.29 захоплень м'ютекса на дію проти 1.40 в а)), а навантаження рівномірно розподілене між 3 полями (по 33.3% звернень), тоді як за умовою (а) 70% усіх операцій концентруються на одному Полі 1 (40% read 1 + 10% write 1 + 20% string). Завдяки використанню std::shared_mutex 60% читань Поля 1 у сценарії (а) виконуються неблокуюче, тому попри двократний перекіс навантаження на Поле 1 та більшу частку string час сценарію (а) майже зрівнявся з рівномірним розподілом. По-третє, зростання часу при переході від 1 до 2/3 потоків при однаковій сумарній кількості дій (600 тис.) пояснюється тим, що елементарна операція читання/запису одного int триває +- 1 нс, а кожне захоплення shared_lock / lock_guard у багатопотоковому режимі модифікує внутрішній лічильник стану м'ютекса, спричиняючи когерентний обмін кеш-лініями L1/L2 між ядрами процесора; водночас між 2 і 3 потоками час стабілізується завдяки паралельному виконанню форматування std::format поза критичною секцією.

6. Що реально було зроблено самостійно: Самостійно проаналізовано частотний баланс Варіанту 5 з урахуванням вкладу операції string (29:1, 6:1 та 3:1 на користь читання), спроєктовано схему синхронізації класу OptimizedData на трьох вирівняних за кеш-лініями std::shared_mutex, усунено deadlock фіксованим порядком блокувань і скорочено критичну секцію в operator string() (форматування поза замками), реалізовано генератор тестових послідовностей на основі <random>, багатопотоковий бенчмарк (без використання векторів, масивів - за вимогами умови (окрім fields_[3])), підібрано контрприклад розподілу для сценарію в та проведено вимірювання часу в режимі Release (-O3).